



GPUs

The `gpu` directive takes `true` as an argument.

```
## -l gpu=true
```

In addition, to request multiple GPUs you need to include the `-pe gpu` directive as follows:

```
## -pe gpu n
```

Where `n` is the number of GPUs requested. Please note that the `-pe gpu n` directive augments the way the scheduler interprets `tmem` and `h_vmem`. These values will be multiplied by `n`, as the scheduler interprets the values of these directives as being 'per thread' (or per GPU in this case).

NB/ `h_vmem` can be omitted for certain GPU jobs (PyTorch and Tensorflow for example). For example, a job requiring 4G of memory and 2 GPUs would have a submission script containing:

```
## -l tmem=2G
```

```
## -l gpu=true
```

```
## -pe gpu 2
```

```
## -R y
```

and a job requiring 4G of memory and a single GPU:

```
## -l tmem=4G
```

```
## -l gpu=true
```

The environment variable that controls which cards are visible (`CUDA_VISIBLE_DEVICES`) is set for you. Please do not attempt to modify `CUDA_VISIBLE_DEVICES` from within your code, nor specify a GPU device index to use.

There is also a maximum runtime on GPU jobs of **90 days**. If you wish to run jobs for longer than this, please contact us (<https://hpc.cs.ucl.ac.uk/contact-us/>).

The NVIDIA drivers are updated regularly, and are currently at version **450.51**.

All GPUs are now set to persistent mode and compute exclusive mode.

Interactive Sessions (qssh)

You no longer have to run the `CUDA_VISIBILITY.sh` script when running interactive/test sessions. The scheduler now handles all environment variables for you, and the same

applies with respect to manually setting the `CUDA_VISIBLE_DEVICES` variable.

UCL Computer Science (<https://www.cs.ucl.ac.uk/>) Technical Support Group (<https://tsg.cs.ucl.ac.uk/>) Copyright © 1972-2023

UCL (<https://www.ucl.ac.uk/>) Unite Theme (<http://colorlib.com/wp/unite>) powered by WordPress (<http://wordpress.org>).

The changes to the flags for requesting GPUs apply to requesting interactive sessions. You can combine the `-pe gpu n` directive with a `qcrsh` request to test interactively across multiple GPUs:

```
qcrsh -l tmem=4G,gpu=true,h_rt=3600 -pe gpu 2
```

There is a fast test queue for interactive work with a short runtime (up to one hour) to enable you to test code quickly without having to wait:

```
qcrsh -l tmem=14G,gpu=true,h_rt=0:30:0
```

Additionally, please only start up to **two qcrsh sessions per user** (except those of you with group GPU nodes where you can request as many sessions as there are slots available).

Specifying GPUs

Specifying a GPU (especially higher end ones) can vastly increase your queue time, and is unnecessary in most cases.

To request a specific GPU, you can add the following to your submission scripts:

```
#$ -l gpu=true,gpu_type=<type>
```

Where type is one of the following:

gtx1080ti for the GTX 1080ti cards (11GB).

titanx for the Titan X cards (12GB).

rtx2080 for the RTX 2080 cards (8GB).

rtx2080ti for the RTX 2080ti cards (11GB).

rtx3090 for the RTX 3090 cards (24GB).

rtx4070ti for the RTX 4070ti Super cards (16GB).

rtx4090 for the RTX 4090 cards (24GB).

p100 for the P100 cards [restricted access only] (16GB).

v100 for the V100 cards [restricted access only] (16GB).

rtx6000 for the RTX 6000 cards [restricted access only] (24GB).

rtx8000 for the RTX 8000 cards [restricted access only] (48GB).

l40s for the L40S cards (46GB).

a100 for the A100 cards [restricted access only] (40GB).

a100_80 for the A100 80GB variant cards [restricted access only] (80GB).

a100_dgx for the A100 cards housed in DGXs [restricted access only] (40GB).

h100 for the H100 cards [restricted access only] (80GB).

So, for example:

```
#$ -l gpu=true,gpu_type=gtx1080ti
```

will attempt to schedule you only on nodes with a 1080ti on them.

UCL Computer Science (<https://www.cs.ucl.ac.uk/>) Technical Support Group (<https://tsg.cs.ucl.ac.uk/>) Copyright © 1972-2023

UCL (<https://www.ucl.ac.uk/>) Unite Theme (<http://colorlib.com/wp/unite>) powered by WordPress (<http://wordpress.org>).

You can also use logical operators to combine multiple types, for example if you wanted only cards with 12GB of memory:

```
#$ -l gpu=true,gpu_type=(titanxp|titanx)
```

Or if you wanted to omit all cards with less than 12GB of memory:

```
#$ -l gpu=true,gpu_type=!(gtx1080ti|rtx2080ti)
```

NVIDIA CUDA/cuDNN

Versions of CUDA are installed in the default location on the GPU nodes of `/usr/local/cuda-<version>` with a symlink from `/usr/local/cuda` to the most current version. NB/ cuDNN is NOT installed on the `/usr/local/` version of CUDA. If you wish to use cuDNN there are also all available within the CUDA versions installed in `/share/apps/`. Each CUDA directory has its corresponding cuDNN versions installed within it.

CUDA version	cuDNN version
7.5	5.1
	6.0
8.0	5.1
	6.0
	7.0.4
9.0	7.0.5
	7.0.4
	7.0.5
9.1	7.0.5
9.2	7.2.1
10.0	7.4.2
	7.6.3
10.1	7.5.0

Torch DDP

There are times where DistributedDataParallel (DDP) calls will hang when using a combination of NCCL backend and PyTorch, especially on cards with NVLinks across them. Try adding the following:

```
export NCCL_P2P_DISABLE=1
```

to your submission scripts before you call `python3`. If you are encountering further problems, please contact us.

UCL Computer Science (<https://www.cs.ucl.ac.uk/>) Technical Support Group (<https://tsg.cs.ucl.ac.uk/>) Copyright © 1972-2023

UCL (<https://www.ucl.ac.uk/>) Unite Theme (<http://colorlib.com/wp/unite>) powered by WordPress (<http://wordpress.org>).

